Name: **Fulekar Hitali Sameer**

Roll No.: **23103051**

CSE 6th sem G2(A)

Task 1: Read, Display, Save Copy

• Objective: Confirm your program can read an image correctly. • Steps to do • Read input.jpg • Display it in a window • Print: • Width, Height • Channels (1 or 3) • Save a copy as: outputs/copy.png • Expected output • Image window opens • Console prints size (example: 1920×1080) and channel info

```
from google.colab import files
uploaded = files.upload()
```

Choose Files image sample.png

image sample.png(image/png) - 195446 bytes, last modified: 2/2/2026 - 100% done

```
Saving image sample.png to image sample (4).png
```

```
import cv2
import os
import sys
from google.colab.patches import cv2_imshow

def main():

    input_path = "/content/image sample.png"
    output_dir = "outputs"
    output_path = os.path.join(output_dir, "copy.png")


    img = cv2.imread(input_path, cv2.IMREAD_UNCHANGED)


    height, width = img.shape[:2]
    channels = 1 if len(img.shape) == 2 else img.shape[2]


    print(f"Width* height : {width} * {height} ")
    print(f"Channels: {channels}")


    cv2_imshow(img)

    os.makedirs(output_dir, exist_ok=True)


    if cv2.imwrite(output_path, img):
        print(f"Image saved as '{output_path}'")
    else:
        print("Error: Failed to save the image.")

if __name__ == "__main__":
    main()
```

```
Width* height : 921 * 1033
Channels: 4
```


```
Image saved as 'outputs/copy.png'
```

Task 2: Objective: Understand image as a pixel matrix. Steps • Convert image to grayscale • Display grayscale image • Save: outputs/gray.png • Print: • grayscale image shape (H, W) • value of one sample pixel (example: pixel at [50,50]) Observation • Grayscale values range from 0 (black) to 255 (white)

```
def main():

    input_path = "/content/image sample.png"
    output_dir = "outputs"
    output_path = os.path.join(output_dir, "gray.png")


    img = cv2.imread(input_path, cv2.IMREAD_UNCHANGED)

    gray_image = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)



    cv2_imshow(gray_image)


    height, width = gray_image.shape[:2]
```

```
    x, y = 50, 50
    pixel_value = gray_image[y,x]


    print(f"Width* height : {width} * {height} ")
    print(f"pixel at {x} , {y}: {pixel_value} ")

    if cv2.imwrite(output_path, gray_image):
        print(f"Image saved as '{output_path}'")
    else:
        print("Error: Failed to save the image.")

if __name__ == "__main__":
    main()
```


```
Width* height : 921 * 1033
pixel at 50 , 50: 0
Image saved as 'outputs/gray.png'
```

Task 3: Perform these operations and save each output: A) Resize • Resize to 50% • Save: outputs/resize_half.png B) Crop • Crop a 200×200 region from center • Save: outputs/crop.png

```
def main():

    input_path = "/content/image sample.png"
```

```
    output_dir = "outputs"
    output_path = os.path.join(output_dir, "resize_half.png")


    img = cv2.imread(input_path, cv2.IMREAD_UNCHANGED)

    resized_img = cv2.resize(img, (img.shape[1]//2 , img.shape[0]//2))


    if cv2.imwrite(output_path, resized_img):
        print(f"Image saved as '{output_path}'")
    else:
        print("Error: Failed to save the image.")


if __name__ == "__main__":
    main()
```

```
Image saved as 'outputs/resize_half.png'
```

```
import cv2
import os
from google.colab.patches import cv2_imshow

def main():

    input_path = "/content/image sample.png"
    output_dir = "outputs"

    img = cv2.imread(input_path)

    height, width = img.shape[:2]


    center_x = width // 2
    center_y = height // 2


    crop_size = 200


    x1 = center_x - crop_size // 2
    y1 = center_y - crop_size // 2
    x2 = x1 + crop_size
    y2 = y1 + crop_size


    cropped_img = img[y1:y2, x1:x2]

    output_path = os.path.join(output_dir, "crop.png")

    if cv2.imwrite(output_path, cropped_img):
        print(f"Cropped image saved as '{output_path}'")
    else:
        print("Error: Failed to save the image.")


if __name__ == "__main__":
    main()
```

```
Cropped image saved as 'outputs/crop.png'
```

C) Flip • Horizontal mirror flip • Save: outputs/flip_h.png D) Rotate • Rotate by 90° • Save: outputs/rotate_90.png • Tip: After saving each file, quickly open it to verify.

```
def main():

    input_path = "/content/image sample.png"
    output_dir = "outputs"
    output_path = os.path.join(output_dir, "flipped_img.png")



    img = cv2.imread(input_path, cv2.IMREAD_UNCHANGED)

    flipped_img= cv2.flip(img,1)

    if cv2.imwrite(output_path, flipped_img):
        print(f"Image saved as '{output_path}'")
```

```
    else:
        print("Error: Failed to save the image.")



if __name__ == "__main__":
    main()
```

```
Image saved as 'outputs/flipped_img.png'
```

```
import cv2
import os
from google.colab.patches import cv2_imshow

def main():

    input_path = "/content/image sample.png"
    output_dir = "outputs"



    output_path = os.path.join(output_dir, "rotate_90.png")

    img = cv2.imread(input_path, cv2.IMREAD_UNCHANGED)


    rotate_90 = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)

    if cv2.imwrite(output_path, rotate_90):
        print(f"Image saved as '{output_path}'")
    else:
        print("Error: Failed to save the image.")


if __name__ == "__main__":
    main()
```

```
Image saved as 'outputs/rotate_90.png'
```

Task 4: A) Brightness • Increase brightness (+) • Decrease brightness (−) • Save: • outputs/bright_plus.png • outputs/bright_minus.png

B) Contrast • Increase contrast • Save: outputs/contrast_high.png C) Negative (Inversion) • Invert pixel values (dark becomes light) • Save: outputs/negative.png

```
import cv2
import os
from google.colab.patches import cv2_imshow
import numpy as np

def main():

    input_path = "/content/image sample.png"
    output_dir = "outputs"



    img = cv2.imread(input_path)

    if img is None:
        print("Error: Image not found!")
        return


    bright_plus = cv2.add(img, 50)
    bright_plus_path = os.path.join(output_dir, "bright_plus.png")
    cv2.imwrite(bright_plus_path, bright_plus)


    bright_minus = cv2.subtract(img, 50)
    bright_minus_path = os.path.join(output_dir, "bright_minus.png")
    cv2.imwrite(bright_minus_path, bright_minus)



    alpha = 1.5
    beta = 0

    contrast_high = cv2.convertScaleAbs(img, alpha=alpha, beta=beta)
    contrast_path = os.path.join(output_dir, "contrast_high.png")
```

```
    cv2.imwrite(contrast_path, contrast_high)



    negative = 255 - img
    negative_path = os.path.join(output_dir, "negative.png")
    cv2.imwrite(negative_path, negative)


    print("All images saved successfully in outputs folder!")


if __name__ == "__main__":
    main()
```

```
All images saved successfully in outputs folder!
```

Task 5: Objective: Convert grayscale to black/white image. Steps • Use grayscale image • Apply simple threshold (example threshold value: 127) • Save: outputs/thresh_simple.png Observation • Output becomes binary-like: • 0 = black • 255 = white

```
import cv2
import os
from google.colab.patches import cv2_imshow

def main():

    input_path = "/content/image sample.png"
    output_dir = "outputs"

    img = cv2.imread(input_path)

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)


    threshold_value = 127
    max_value = 255

    ret, thresh_img = cv2.threshold(gray, threshold_value, max_value, cv2.THRESH_BINARY)

    output_path = os.path.join(output_dir, "thresh_simple.png")
    cv2.imwrite(output_path, thresh_img)

    print("Threshold image saved as:", output_path)


if __name__ == "__main__":
    main()
```

```
Threshold image saved as: outputs/thresh_simple.png
```

Task 6: Objective: Reduce noise and make contours/edges better. Apply any 3 (minimum) and save outputs: • Averaging Blur → outputs/blur_avg.png • Gaussian Blur → outputs/blur_gaussian.png • Median Blur → outputs/blur_median.png • (Optional) Bilateral Filter → outputs/blur_bilateral.png

```
import cv2
import os
from google.colab.patches import cv2_imshow

def main():

    input_path = "/content/image sample.png"
    output_dir = "outputs"

    img = cv2.imread(input_path)

    blur_avg = cv2.blur(img, (5,5))
    cv2.imwrite(os.path.join(output_dir, "blur_avg.png"), blur_avg)



    blur_gaussian = cv2.GaussianBlur(img, (5,5), 0)
    cv2.imwrite(os.path.join(output_dir, "blur_gaussian.png"), blur_gaussian)


    blur_median = cv2.medianBlur(img, 5)
    cv2.imwrite(os.path.join(output_dir, "blur_median.png"), blur_median)
```

```
    blur_bilateral = cv2.bilateralFilter(img, 9, 75, 75)
    cv2.imwrite(os.path.join(output_dir, "blur_bilateral.png"), blur_bilateral)


    print("All blur images saved successfully!")


if __name__ == "__main__":
    main()
```

```
All blur images saved successfully!
```

1. Which blur keeps edges more clear? **Bilateral Filter** keeps edges the most clear, because it smooths only similar pixels and avoids blurring across edges.
2. Which blur makes the image most smooth? **Gaussian Blur **makes the image most smooth, as it spreads pixel values evenly and produces soft, uniform smoothing.

Task 7: Objective: Detect boundaries of objects. Steps • Use grayscale or blurred grayscale • Apply Canny Edge Detection • Save: outputs/edges.png • Expected • Only outlines (edges) appear in white over black background

```
import cv2
import os
from google.colab.patches import cv2_imshow

def main():

    input_path = "/content/image sample.png"
    output_dir = "outputs"

    img = cv2.imread(input_path)

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)


    blurred = cv2.GaussianBlur(gray, (5,5), 0)


    edges = cv2.Canny(blurred, threshold1=50, threshold2=150)


    output_path = os.path.join(output_dir, "edges.png")
    cv2.imwrite(output_path, edges)

    print("Edge detected image saved as:", output_path)


if __name__ == "__main__":
    main()
```

```
Edge detected image saved as: outputs/edges.png
```

Task 8: Objective: Find object shapes and draw boundaries. Pipeline • Grayscale → Blur → Threshold • Find contours from threshold image • Draw contours on original image copy • Save: • outputs/binary_for_contours.png (threshold used) • outputs/contours.png (final drawn contour image) • Print number of contours found Expected • Object borders highlighted with colored lines

```
import cv2
import os
from google.colab.patches import cv2_imshow

def main():

    input_path = "/content/image sample.png"
    output_dir = "outputs"
    img = cv2.imread(input_path)

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)


    blurred = cv2.GaussianBlur(gray, (5,5), 0)


    ret, binary = cv2.threshold(blurred, 127, 255, cv2.THRESH_BINARY)
```

```
    binary_path = os.path.join(output_dir, "binary_for_contours.png")
    cv2.imwrite(binary_path, binary)


    contours, hierarchy = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    print("Number of contours found:", len(contours))


    contour_img = img.copy()

    cv2.drawContours(contour_img, contours, -1, (0, 255, 0), 2)

    contour_path = os.path.join(output_dir, "contours.png")
    cv2.imwrite(contour_path, contour_img)

    print("Images saved successfully!")


if __name__ == "__main__":
    main()
```

```
Number of contours found: 38
Images saved successfully!
```

```
!ls /content/outputs
```

```
binary_for_contours.png  bright_minus.png   crop.png         resize_half.png
blur_avg.png             bright_plus.png    edges.png        rotate_90.png
blur_bilateral.png       contours.png       flipped_img.png  thresh_simple.png
blur_gaussian.png        contrast_high.png  gray.png
blur_median.png          copy.png           negative.png
```